

Universidade Federal de Uberlândia
Faculdade de Computação

Engenharia de Software
Prof. Fabiano Azevedo Dorça

Testes de Software
Parte 01

Testes

- O teste de software é um elemento de um tema mais amplo, muitas vezes conhecido como verificação e validação (V&V).
 - Verificação refere-se ao conjunto de tarefas que garantem que o software execute corretamente uma função específica.
 - Validação refere-se ao conjunto de tarefas que asseguram que o software foi criado segundo os requisitos do cliente.

Testes

- **Verificação**: “Estamos criando o produto corretamente?”
 - Funciona conforme especificado?
- **Validação**: “Estamos criando o produto certo?”
 - Atende o que o usuário deseja?

Testes

- Para **evitar que erros cheguem aos usuários** finais e causem prejuízos de valor incalculável, é
 - **fundamental** introduzir atividades de teste em projetos de desenvolvimento de software.
- É uma das **práticas de programação mais valorizadas** hoje em dia, em qualquer tipo de software.
- É uma das práticas que sofreram mais transformações nos anos recentes.

Testes

- Desenvolvimento era em **cascata**,
 - os testes **ocorriam em uma fase separada**, após as fases de levantamento de requisitos, análise, projeto e codificação.
 - existia uma **equipe separada de testes**, responsável por verificar se a implementação atendia aos requisitos do sistema.

Testes

- Para isto, **frequentemente os testes eram manuais,**
 - uma **pessoa usava o sistema**, informava dados de entrada e verificava se as saídas eram aquelas esperadas.
 - assim, o objetivo de tais testes era **apenas detectar bugs, antes que o sistema entrasse em produção.**

Testes

- Com **métodos ágeis**, a prática de testes de software foi profundamente reformulada:
 - **Grande parte dos testes passou a ser automatizada:**
 - Além de implementar **as classes de um sistema**, os desenvolvedores passaram a implementar também **código para testar tais classes**.
 - Assim, os programas **tornaram-se auto-testáveis**.

Testes

- Testes não são mais implementados após todas as classes de um sistema ficarem prontas.
 - Muitas vezes, eles são **implementados até mesmo antes dessas classes.**
- **Não existem mais grandes equipes de testes** — ou elas são responsáveis por testes específicos.
 - Em vez disso, **o desenvolvedor que implementa uma classe também deve implementar os seus testes.**

Testes

- Testes não são mais um instrumento **exclusivo para detecção de bugs.**
 - testes ganharam novas funções, como **verificar se uma classe continuará funcionando** após um bug ser corrigido em uma outra parte do sistema.
- Testes são também usados como **documentação para o código de produção.**

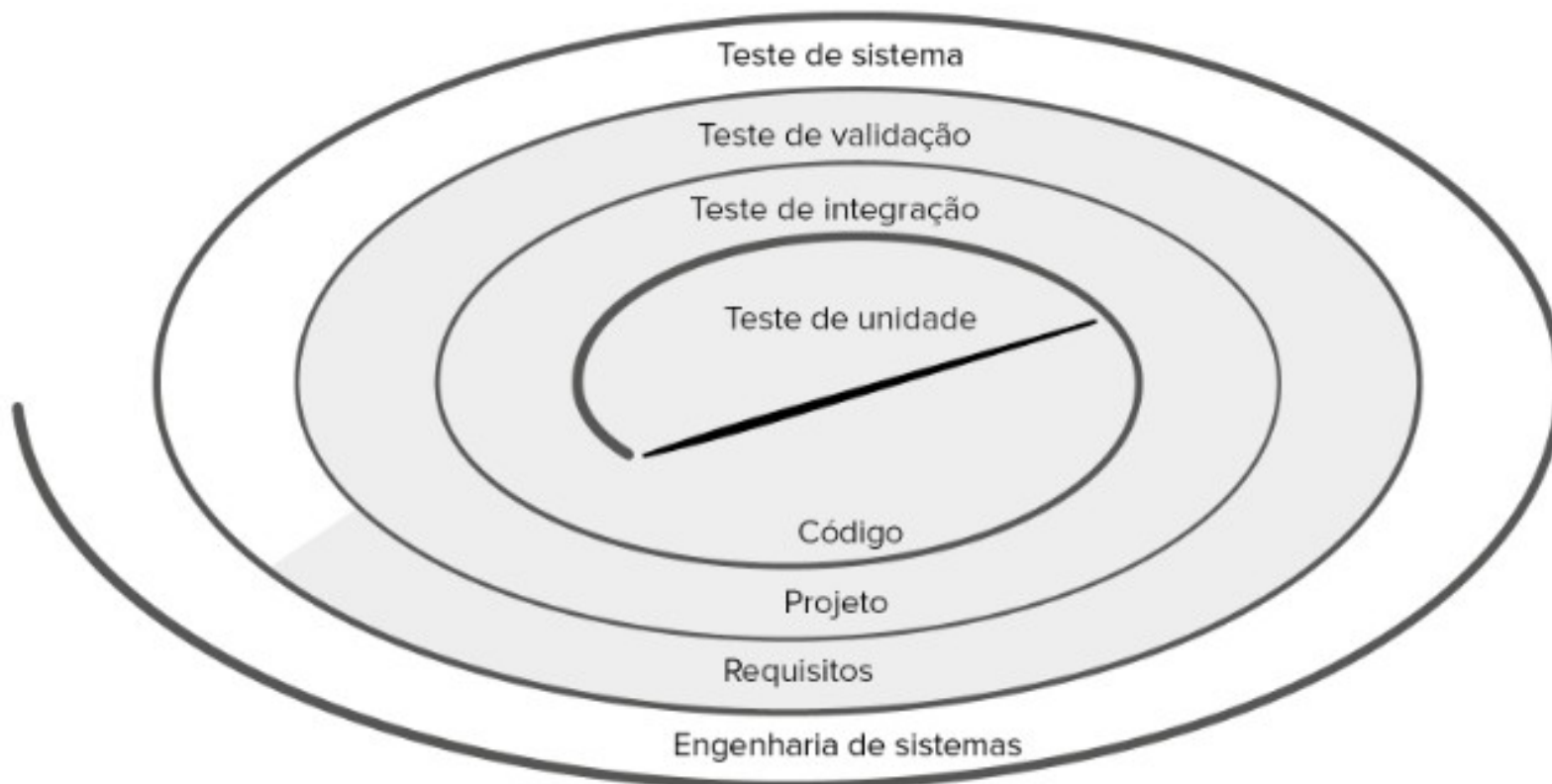
Essas transformações tornaram testes uma das **práticas de programação mais valorizadas** em desenvolvimento moderno de software.

Testes

- A qualidade é incorporada ao software por meio do processo de engenharia de software,
 - *os testes não podem ser simplesmente aplicados no final do processo para consertar tudo.*
 - A aplicação correta de métodos e ferramentas, de revisões técnicas eficazes e de um sólido gerenciamento e avaliação *conduzem à qualidade que é confirmada durante o teste.*

Testes

- O processo de software pode ser visto como a espiral



Testes

- ***Inicialmente,***
 - a engenharia de sistemas define o papel do software e passa à análise dos requisitos de software, na qual são estabelecidos o domínio da informação, a função, o comportamento, o desempenho, as restrições e os critérios de validação para o software.
- ***Deslocando-se para o interior da espiral,***
 - chega-se ao projeto e, por fim, à codificação. Para desenvolver software de computador, percorre-se a espiral para o interior ao longo de linhas que indicam a diminuição do nível de abstração a cada volta.

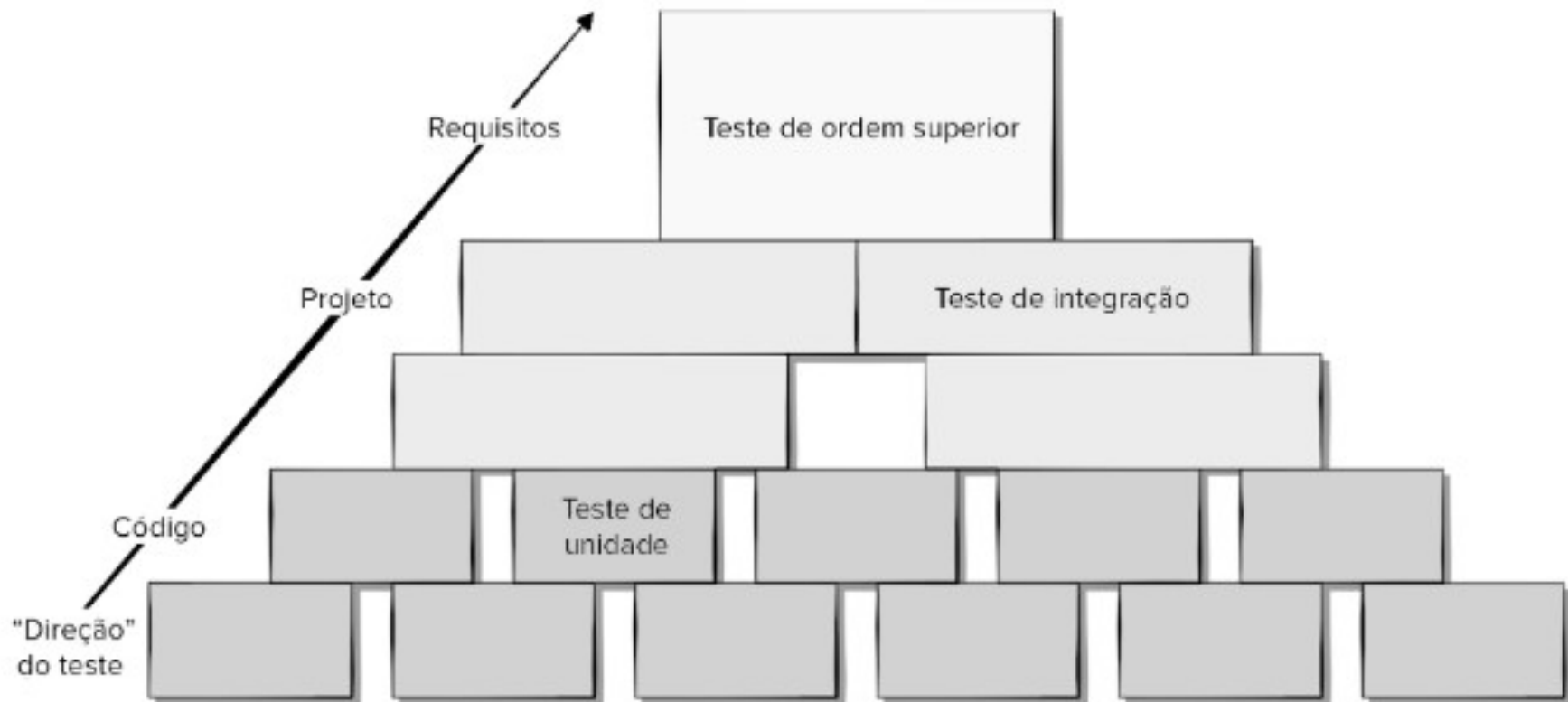
Testes

- O **teste de unidade** começa no centro da espiral e se concentra em cada unidade (p. ex., componente, classe ou objeto de conteúdo de WebApp) do software, conforme implementado no código-fonte.
- O teste prossegue movendo-se em direção ao exterior da espiral, passando pelo **teste de integração**, em que o foco está no projeto e na construção da arquitetura de software.

Testes

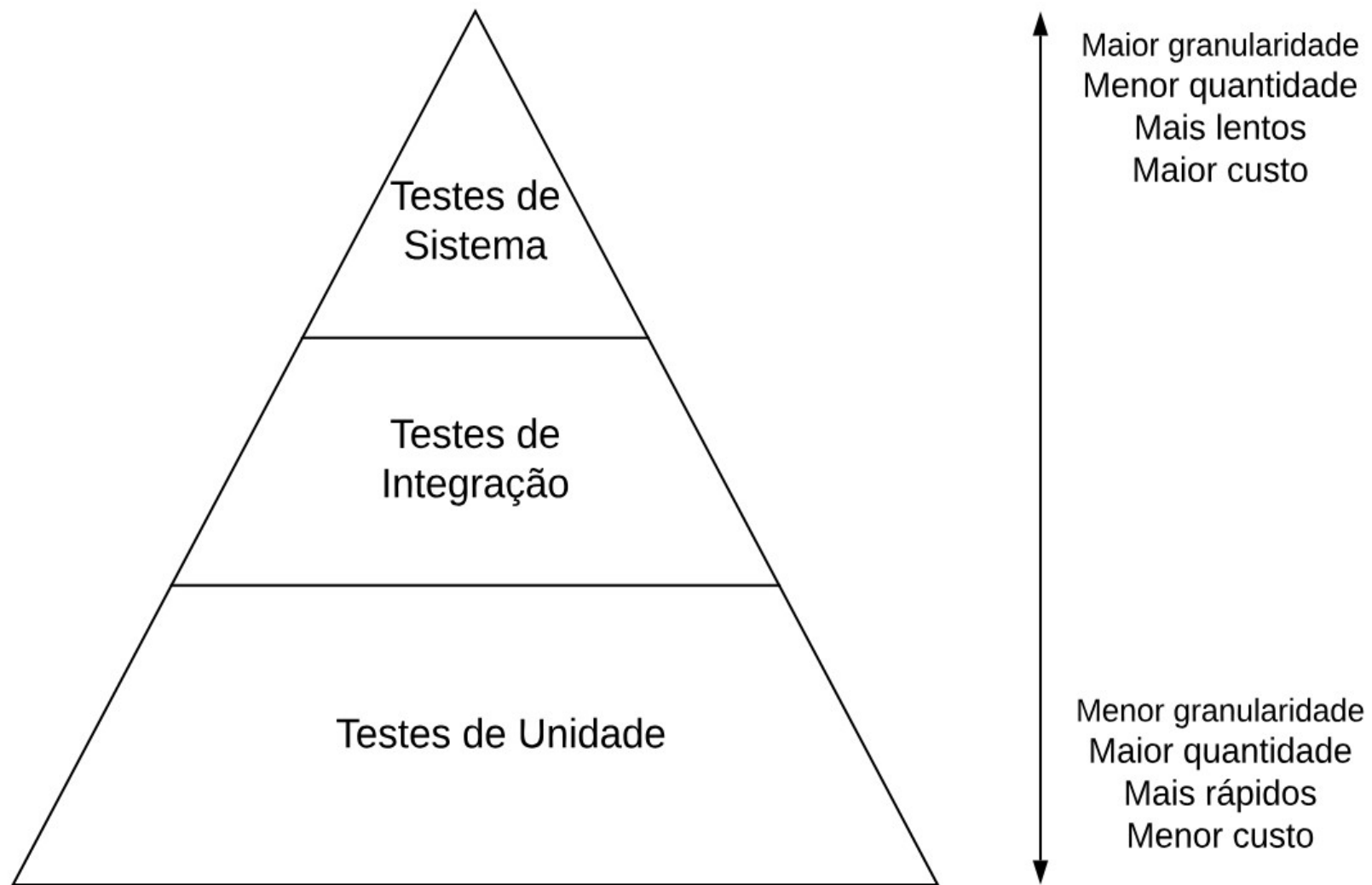
- O **teste de validação**, em que requisitos estabelecidos como parte dos requisitos de modelagem são validados em relação ao software criado.
- Por fim, chegamos ao **teste do sistema**, no qual o software e outros elementos são testados como um todo.
- Para testar um software de computador, percorre-se a espiral em direção ao seu exterior, ao longo de linhas que indicam o escopo do teste a cada volta.

Testes



Testes

- Testes de acordo com sua **granularidade**

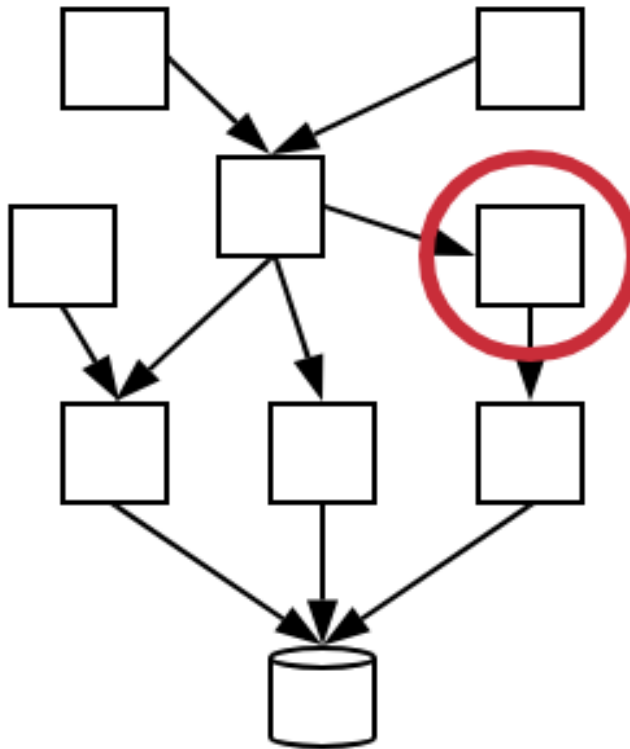


Testes

- Testes de unidade **verificam automaticamente pequenas partes de um código,**
 - normalmente **uma classe apenas**
- Eles formam a **base da pirâmide**, ou seja, a maior parte dos testes estão nessa categoria.
- **Testes de unidade são simples**, mais fáceis de implementar e executam rapidamente.

Testes

- Escopo de testes **de unidade**

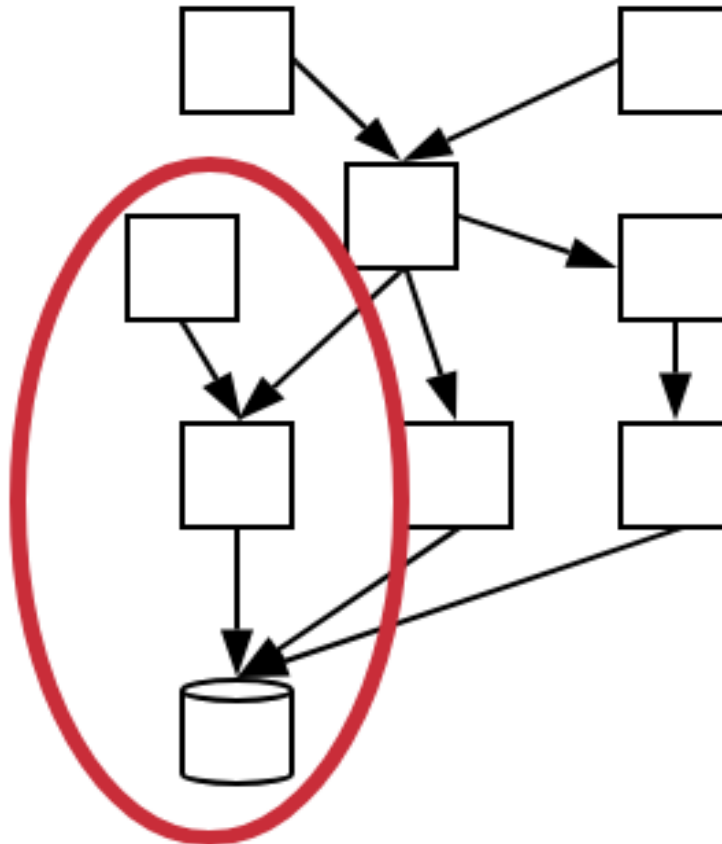


Testes

- Testes de **integração** ou testes de serviços,
 - que verificam uma **funcionalidade ou transação completa** de um sistema.
 - são testes que usam **diversas classes**, de pacotes distintos, e podem ainda testar componentes externos, como bancos de dados.
 - Testes de integração **demandam mais esforço** para serem implementados e executam de forma mais lenta.

Testes

- Escopo de testes de integração



Testes

- **Testes de Integração**

- Com testes de integração — também chamados de testes de serviços — subimos para um nível intermediário da pirâmide de testes
- Assim, o objetivo deixa de ser o teste de uma unidade pequena de código, como uma classe apenas.

Em vez disso, testes de integração exercitam um serviço completo, isto é, uma funcionalidade de maior granularidade do sistema.

–

Testes

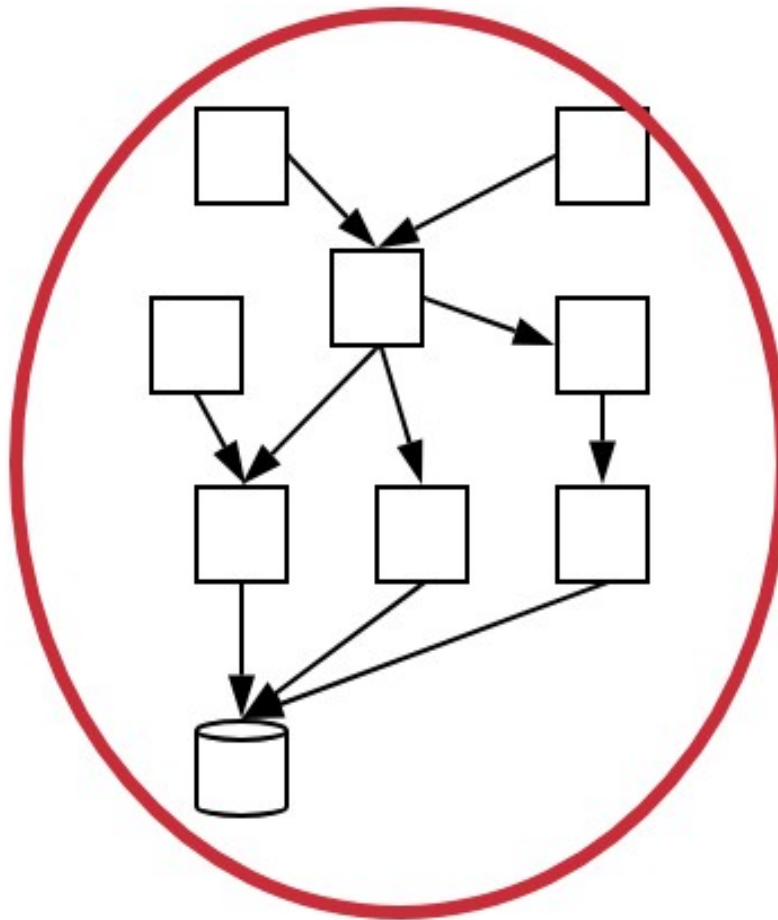
- Por isso, eles **envolvem mais classes**, às vezes de pacotes distintos.
- Também **envolvem dependências e sistemas reais**, como bancos de dados e serviços remotos.
- Como são **testes maiores, eles levam mais tempo para executar** e, conseqüentemente, são chamados com menor frequência.
- Ele é um **teste de integração, quando a classe é testada com dependências reais**.

Testes

- **Testes de sistema**, também chamados de testes de interface com o usuário.
 - simulam, da forma mais fiel possível, **uma sessão de uso do sistema por um usuário real.**
 - são testes de ponta a ponta (end-to-end), **eles são mais caros, mais lentos e menos numerosos.**
- Testes de interface **costumam ser também frágeis,**
 - isto é, **mínimas alterações nos componentes da interface podem demandar modificações nesses testes.**

Testes

- Escopo de testes de sistema

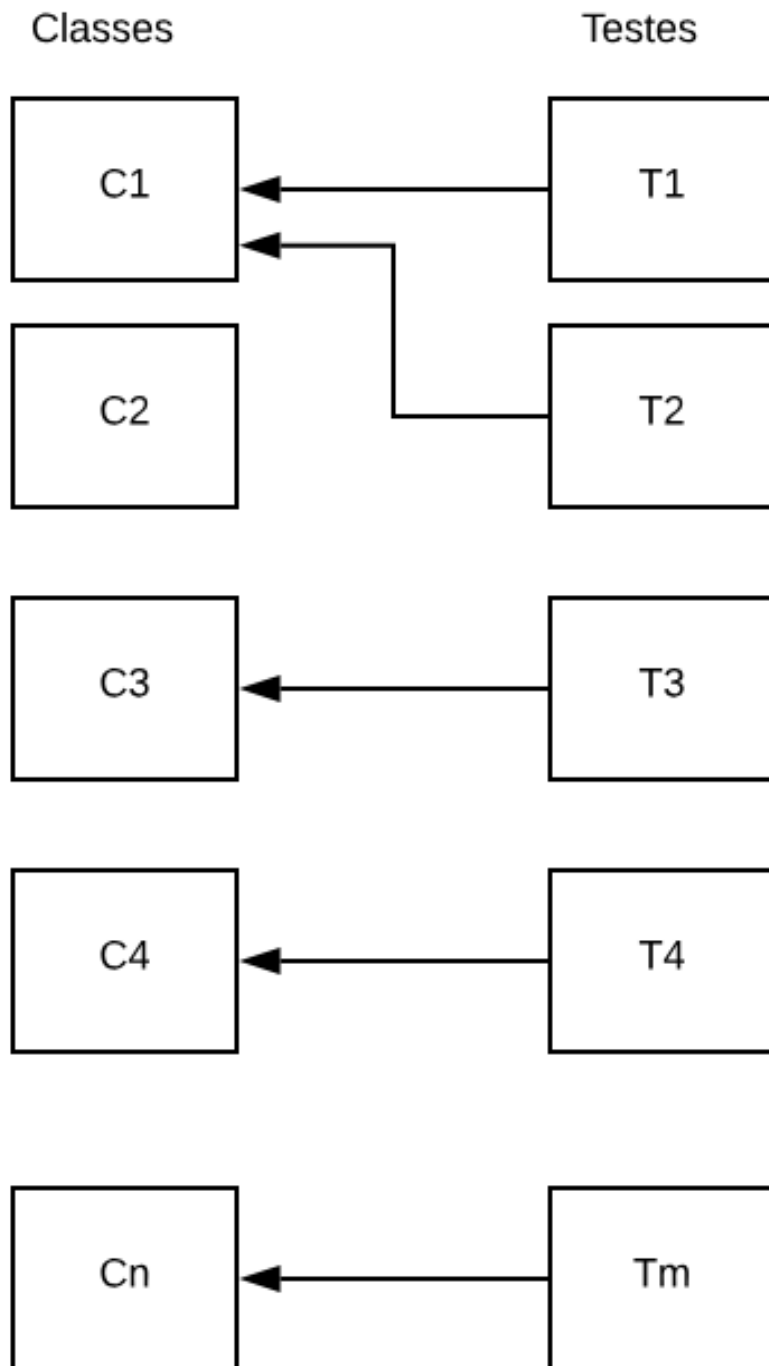


Testes

- Testes de **Unidade**
 - Testes de unidade **são testes automatizados** de pequenas unidades de código, normalmente classes, as quais são testadas de forma isolada do restante do sistema.
 - Um teste de unidade **é um programa que chama métodos de uma classe** e verifica se eles retornam os resultados esperados.

Testes

- Assim, quando se usa testes de unidades, o **código de um sistema pode ser dividido em dois grupos:**
 - um **conjunto de classes** — que implementam os requisitos do sistema —
 - um **conjunto de testes** – classes que implementam testes



Como pode ser observado, **não existe uma correspondência de 1 para 1** entre classes e testes.

Por exemplo, uma classe pode ter mais de um teste.

É o caso da classe C1, que é testada por T1 e T2.

Provavelmente, isso ocorre porque C1 é uma classe importante, que precisa ser testada em diferentes contextos.

C2 não possui testes, ou porque os desenvolvedores esqueceram de implementar ou porque ela é uma classe menos importante.

Testes

Dentre as práticas de desenvolvimento propostas originalmente por **métodos ágeis**, *testes de unidade é provavelmente a que alcançou o maior impacto e que é mais largamente usada*

- Testes de unidade **ajudam na documentação e especificação do código de produção.**

Por isso, muitas vezes, antes de manter um código com o qual ele não tenha familiaridade, um desenvolvedor começa analisando os seus testes.

Testes

- Testes x Requisitos
 - A **elaboração dos casos de testes** a partir da **especificação de requisitos** é uma ferramenta muito poderosa de **verificação de requisitos**, ajudando a melhorar a qualidade, **tanto dos testes quanto dos requisitos..**

Testes

- **Testes de unidade** funcionam como uma **rede de proteção** contra **regressões no código.**
- **Regressão**
 - ocorre quando uma **modificação** realizada no código de um sistema
 - seja para corrigir um bug, implementar uma nova funcionalidade ou realizar uma refatoração
 - **acaba por introduzir um bug ou outro problema semelhante no código.**

Testes

- **Dizemos que o código regrediu** porque algo que estava funcionando deixou de funcionar após a mudança que foi realizada.
- **Regressões são mais raras quando se tem bons testes.**

Testes

- Para isso, após uma mudança o desenvolvedor deve rodar a suíte de testes.
- Se a mudança tiver introduzido alguma regressão, existe uma boa chance de que ela seja detectada pelos testes.
- Ou seja, antes da mudança os testes estavam passando, mas após a mudança algum teste começou a falhar.

Testes

- **Teste de regressão**
 - Usado para confirmar que o programa mais recente ou as alterações relacionadas ao código **não afetaram os recursos existentes de maneira adversa.**
 - Seu objetivo é garantir que **nenhum defeito foi acrescentado** ao sistema após sua evolução.
- Durante o **teste de regressão, é verificado se o software funciona bem em relação às novas alterações e correções de bugs.**

Testes

- O **teste de regressão trata principalmente da reexecução** dos casos de teste já executados para confirmar se o aplicativo está funcionando adequadamente.
- O teste de regressão deve ser **realizado mesmo nas menores alterações no aplicativo** para analisar o impacto da alteração.
- Caso a automação não seja usada no projeto, **pode ser muito tedioso** executar um único caso de teste repetidamente.

Testes

- **Testes de Sistema**

- Testes de sistema estão posicionados no topo da pirâmide de testes.
- **Trata-se de testes que simulam o uso de um sistema por um usuário real.**
- Testes de sistema são também chamados de **testes ponta-a-ponta (end-to-end)** ou então **testes de interfaces.**
- São os testes mais caros, que demandam maior esforço para implementação e que executam em mais tempo.

• Testes de Aceitação

- São testes realizados pelo cliente, com dados do cliente.
- Os resultados desses testes irão determinar se o cliente está de acordo ou não com a implementação realizada.
- Se estiver de acordo, o sistema pode entrar em produção.
- Se não estiver de acordo, os devidos ajustes devem ser realizados.

Quando se usa métodos ágeis, uma história somente é considerada completa após passar por testes de aceitação, realizados pelos usuários, ao final de um sprint.

Testes

- **Testes de aceitação (validação)**

- Eles não constituem exclusivamente uma atividade de **verificação** (como os testes anteriores), mas também uma atividade de **validação** do sistema.

- **verificação** testa se fizemos o sistema corretamente, isto é, de acordo com a sua especificação e/ou requisitos.
 - **validação** testa se fizemos o sistema correto, isto é, aquele que o cliente pediu e precisa.

Testes

- **Testes de aceitação** podem ser divididos em duas fases.
 - **Testes alfa** são ***realizados com alguns usuários, mas em um ambiente controlado***, como a própria máquina do desenvolvedor.
 - Se o sistema for aprovado nos testes alfa, **pode-se realizar um teste com um grupo maior de usuários** e *não mais em um ambiente controlado*.
 - Esses testes são chamados de **testes beta**.

Testes

- **Testes de Requisitos Não-Funcionais**
 - É possível realizar também testes para verificar ou validar requisitos não-funcionais.
 - Por exemplo, existem ferramentas que permitem a realização de **testes de desempenho**, para verificar o comportamento de um sistema com alguma carga.
 - Uma empresa de comércio eletrônico pode usar uma dessas ferramentas para simular o desempenho de seu site durante um grande evento, como uma Black-Friday, por exemplo.

Testes

- **Testes de usabilidade** são **usados para avaliar a interface do sistema** e, normalmente, envolvem a observação de usuários reais usando o sistema.
- **Testes de falhas** simulam eventos anormais em um sistema, por exemplo a queda de alguns serviços ou mesmo de um data-center inteiro.

Testes

- Dúvidas??